DevOps Shack
devopsshack.com | @devopsshack

GitLab
Groups & Projects
Deep Dive

Structuring GitLab for Real-World Teams

CI/CD • RBAC • Group Hierarchies • Best Practices

1. Quick Recap & Context

Before diving deep, let's align on the fundamentals that everything else builds on.

Term	One-Line Definition
GitLab	A complete DevOps platform — code, CI/CD, security, and collaboration in one place
Project	The execution unit — where your code lives, pipelines run, and deployments happen
Group	The organizational unit — how teams, permissions, and projects are structured

Today we go deeper into:

- Structuring GitLab for real-world teams
- Avoiding common mistakes early in your setup
- Understanding how Projects and Groups interact at scale

2. Deep Dive: Projects

A Project in GitLab is far more than just a code repository. It is the complete execution unit for your DevOps workflow.

What Is a Project?

At its core, every GitLab Project contains:

Project = Git Repo + CI/CD + Issue Tracking + Deployments
A Git repository with branches, commits, and a default branch (main/master)
A CI/CD pipeline execution engine driven by .gitlab-ci.yml
An issue tracker for bugs, features, and work items
Merge Request workflows for code review and collaboration
Environment definitions for Dev, Staging, and Production

Project Components in Detail

1. Repository

- Stores all source code, configuration files, and scripts
- Supports branches, tags, and commit history
- Default branch is typically main or master

2. CI/CD Pipeline

- Defined by .gitlab-ci.yml in the project root
- Executes Pipelines, Jobs, and Stages automatically on push/merge
- Integrates with container registries, artifact stores, and environments

3. Issues

- Built-in work tracking for bugs, features, and tasks
- Supports labels, milestones, assignees, and weights
- Integrates directly with Merge Requests for traceability

4. Merge Requests (MRs)

- The standard workflow for code review and collaboration
- Can enforce approvals, run pipelines, and block merges on failure
- Linked to issues for full traceability from code to ticket

5. Environments

- Tracks deployments to named environments (dev, staging, prod)
- Provides deployment history, rollback, and live environment URLs
- Can be protected to require manual approval before deploy

Project Types

Type	Description & Examples
Application Project	A single microservice or application (e.g., auth-service, payment-api). Has full CI/CD from build to deploy.
Infrastructure Project	Terraform configs, Helm charts, Kubernetes manifests. Used in GitOps workflows to manage infrastructure as code.
Configuration Project	Shared CI/CD templates, scripts, and reusable configuration files referenced by other projects.

The Golden Rule: One Service = One Project

☑ Best Practice
Each microservice should have its own dedicated project
Each project gets its own pipeline, issues, MRs, and deployment history
This enables independent deployments, clear ownership, and clean CI/CD

⚠ Avoid This
Do NOT put multiple services in a single project
This creates deployment coupling and messy pipeline logic
It makes it impossible to deploy services independently

3. Deep Dive: Groups

If Projects are where work happens, Groups are how that work is organized and managed at scale.

What Is a Group?

A Group is a namespace that acts as a container for Projects and other Subgroups. It is your primary tool for organizing teams, managing permissions, and sharing resources.

Group Features

1. Centralized Access Control

- Add a user to a Group once — they automatically inherit access to all projects inside it
- No need to manually add users project by project
- Permissions flow down through the group hierarchy

2. Subgroups — Nested Hierarchies

Groups can be nested to mirror your organization's structure:

```
company/
  backend/     ← Subgroup: Backend Team
  frontend/    ← Subgroup: Frontend Team
  devops/      ← Subgroup: DevOps/Platform Team
```

- Each subgroup inherits settings from its parent group
- Teams work within their subgroup without affecting others
- Permissions can be overridden at the subgroup level if needed

3. Shared Resources

At the group level you can define resources that all projects in the group can use:

- CI/CD templates — reusable pipeline definitions
- Variables — environment variables available to all child projects
- Runners — GitLab Runners registered at group level for shared compute

Why Groups Matter

Without Groups	With Groups
Access control becomes messy	Add user once, they get access everywhere
Scaling is painful	Hierarchy mirrors org structure naturally
No logical separation	Teams own their namespace
Hard to share resources	Templates, variables, runners shared across projects
Audit is difficult	Clear membership and permission trails

4. Group vs Project — Real Use Case

Let's walk through how a real e-commerce company would structure their GitLab workspace.

Scenario: E-Commerce Company

```
ecommerce/              ← Main Group
  frontend/             ← Subgroup: Frontend Team
    web-app             ← Project: React/Next.js App
  backend/              ← Subgroup: Backend Team
    auth-service        ← Project: Authentication API
    payment-service     ← Project: Payment Processing
    order-service       ← Project: Order Management
  infra/                ← Subgroup: Platform/SRE Team
    terraform           ← Project: Cloud Infrastructure
    kubernetes          ← Project: K8s Manifests & Helm
```

Entity	Role in the Structure
ecommerce (Group)	Top-level namespace. Company-wide CI variables and runner configuration live here.
frontend, backend, infra (Subgroups)	Team-level separation. Each team manages their own projects and access.
auth-service, payment-service etc. (Projects)	Execution units. Each has its own pipeline, issues, MRs, and deployment environments.

Key Insight

Group = Logical separation (who owns what, how teams are organized)

Subgroup = Team-level separation (mirrors org structure, enables delegation)

Project = Execution unit (where pipelines run and code lives)

5. Access Control (RBAC) Deep Dive

Role-Based Access Control is one of the most critical concepts in GitLab for any DevOps or SRE practitioner. Getting this wrong creates security gaps and operational overhead.

GitLab Roles

Role	Access Level	Push Code	Manage Project	Full Control
Guest	View Only	✗	✗	✗
Reporter	Read + Basic	✗	✗	✗
Developer	Contribute	✓	✗	✗
Maintainer	Manage	✓	✓	✗
Owner	Full (Group)	✓	✓	✓

Best Practice: Manage Access at Group Level

☑ Best Practice

Add the backend team to the 'backend' subgroup — they automatically get access to all backend projects

Dev team → Developer role on the app subgroup

SRE team → Maintainer role on the infra subgroup

Security team → Reporter role at the top-level group for visibility without write access

⚠ Avoid This

Do NOT add users individually to each project

This creates massive operational overhead as projects scale

Access becomes inconsistent — some users have access to some projects but not others

Offboarding a team member requires hunting down every individual project assignment

Real-World RBAC Mapping

Team / Role	GitLab Permission Level
Development Team	Developer — can push branches, create MRs, run pipelines
SRE / Platform Team	Maintainer — can manage environments, approve MRs, modify CI settings
Security Team	Reporter — read access to view pipelines, issues, and code without write access
Team Leads / Architects	Maintainer at subgroup level — oversight without needing Owner privileges
Billing / Executive	Guest — view dashboards and issues without touching code

6. Project Visibility & Access

Every project in GitLab has a visibility level that controls who can see and interact with it. Choose carefully — this is a security decision.

Visibility Levels

Level	Who Can See	Use Case
Private	Invited users only	Company/internal projects (recommended)
Internal	All logged-in users	Internal tools & wikis
Public	Anyone on the internet	Open-source projects only

☑ Best Practice

- Always use Private for internal company projects — this is non-negotiable
- Only set a project to Public if you are intentionally open-sourcing it
- Internal is useful for tools that all employees should be able to discover
- Group visibility cascades — a public group with private projects still protects the code

7. Structuring Best Practices

Here is the recommended GitLab structure pattern for real-world teams of any size.

Recommended Pattern

```
company/                ← Root Group
```

```
apps/              ← Subgroup: Application Services
  service-1        ← Project
  service-2        ← Project
  service-3        ← Project
platform/          ← Subgroup: Platform Engineering
  infrastructure   ← Project: Terraform / Cloud
  monitoring       ← Project: Prometheus / Grafana configs
shared/            ← Subgroup: Shared Tooling
  ci-templates     ← Project: Reusable .gitlab-ci.yml includes
  scripts          ← Project: Utility scripts and tools
```

Key Principles

1. **One service = one project** Never group multiple services under a single project repository.
2. **Use subgroups for teams** Mirror your organization's team structure in your GitLab group hierarchy.
3. **Manage access at group level** Never assign individual user permissions per project — it does not scale.
4. **Separate infra and app repos** Infrastructure code in its own subgroup enables GitOps and prevents deploy coupling.

8. Summary

Everything we covered today builds toward one goal: a GitLab workspace that scales with your team, not against it.

Concept	Core Takeaway
Project	Where code and pipelines live. One service = one project. Never combine services.
Group	How teams and projects are organized. Mirrors your org structure.
Subgroup	Team-level separation within a group. Enables delegation and ownership.
RBAC	Always manage at group level. Map team roles, not individuals to projects.
Visibility	Private for all internal work. Public only for intentional open-source.
Structure	apps/ + platform/ + shared/ is the recommended starting pattern.

The Mental Model to Remember
Group = Logical separation (who owns what, how teams are organized)

Project = Execution unit (where pipelines run and code actually lives)

RBAC = Inherited control (set at group, flows down to every project)